CS3230 Additional Notes:

.

AVL Tree Re-Balancing,

.

from [RND-77], pp. 235-243

.

.

E. M. Reingold, J. Nievergelt, Narsingh Deo, "Combinatorial Algorithms: Theory and Practice," Prentice Hall, 1977.

A partial answer to the question about the effect of insertions and deletions on the shape of a binary search tree is now given by the analysis of random trees in Section 6.3.3. If n names are successively inserted by Algorithm 6.8 into an initially empty tree, and if all permutations of the input sequence are equally likely, then this "construction by repeated insertion" produces trees that are random in the sense of Section 6.3.3 with $\beta_i = 1, 1 \leqq i \leqq n$. Hence the average search time in such trees, measured in terms of the number of nodes visited, is $(2 \ln 2) \lg n + O(1)$, approximately 1.4 times longer than the $\lg n + O(1)$ average search time in a completely balanced binary tree of n nodes.

A search time approximately 40% longer than in completely balanced binary trees could be tolerated in most applications, but unconstrained binary search trees are unreliable in practice, since "biased" runs of insertions of names in approximate order are likely to occur and the resulting trees are not random. Moreover, the worst-case behavior is linear in the table size, not logarithmic. The balancing schemes presented in the next three sections guarantee logarithmic search times without requiring any assumptions about the sequence of insertions and deletions.

6.4.2 *Height-Balanced Binary Trees*

The most obvious technique to prevent a dynamic tree from becoming skewed is to keep it as balanced as possible at all times. Thus if a new name B is inserted in the tree shown at left in Figure 6.12, making it less than completely balanced, the tree is restructured as shown. Unfortunately, this transformation generally changes the entire tree: not a single father-son relationship remains unchanged in the example of Figure 6.12. So this restructuring operation, in general, requires an amount of work proportional to n.

The balancing techniques discussed here and in the next section have the property that if a transformation of a tree is required after an insertion or deletion, it can be carried out by *local changes along a single path from root to a leaf*; as we shall see, this requires $O(\log n)$ time. To achieve such flexibility, the trees must be allowed to deviate from completely balanced binary trees, but they do so to a sufficiently small extent that the average search time in them is only slightly larger than that in a completely balanced binary tree.

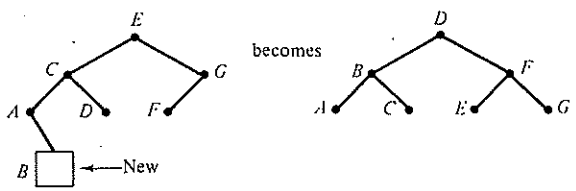


Figure 6.12 Rebalancing a tree completely requires an amount of work proportional to n.

As in Chapter 2, let $h(T)$ denote the height of a binary tree T—that is, the length of the longest path from the root to any leaf. The height of a tree with one single node is 0, and, by convention we consider the height of an empty tree to be -1.

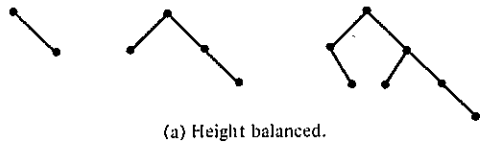
Definition

A binary tree T is *height balanced* if and only if the two subtrees of the root, T_l and T_r, satisfy

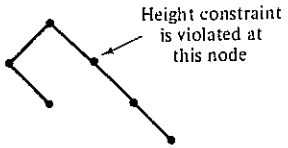
1. $|h(T_l) - h(T_r)| \leqq 1$.
2. T_l and T_r are height balanced.

The empty binary tree, having no root nor subtrees, satisfies these conditions vacuously and hence is height balanced. A tree with a single node is height balanced by our convention about the height of the empty tree. Figure 6.13 shows some height-balanced binary trees and one binary tree that is not. The height constraint prevents height-balanced binary trees from being too far away from a completely balanced binary tree. Indeed, Figure 6.13(a) shows the three "most-skewed" height-balanced binary trees among those of height 1, 2, and 3, respectively.

Despite the fact that height-balanced binary trees can look sparse, the search time that they require is only moderately longer than in completely balanced binary trees. Empirical evidence suggests that the average internal path length of a height-balanced binary tree of n nodes is $n \lg n + O(n)$; consequently, the average time is $\lg n + O(1)$, which differs asymptotically from average search time in completely balanced binary trees by only a constant.



(a) Height balanced.



(b) Not height balanced.

Figure 6.13 Examples of height-balanced binary trees and a non-height-balanced binary tree.

There is no known mathematical analysis of the average search time in height-balanced binary trees of n nodes; it is difficult merely to count the height-balanced binary trees of n nodes (see Exercise 18), let alone analyze their shapes. However, if we consider a worst-case (most-skewed) height-balanced binary tree, we *can* analyze the maximum search time in the tree (the height) and the average successful search time in the tree (the internal path length divided by the number of nodes).

A most-skewed height-balanced binary tree of height h, T_h, has a most-skewed binary tree T_{h-1} of height $h - 1$ as one of its subtrees and a most-skewed binary tree T_{h-2} of height $h - 2$ as the other, as shown in Figure 6.14. The most-skewed height-balanced binary trees shown in Figure 6.13(a) for $h = 1$, 2, and 3 are constructed according to Fibonacci-type rule, starting with an empty tree T_{-1} and a tree T_0 with a single node.

Denoting the number of nodes of T_h by n_h and the internal path length of T_h by p_h, we obtain the recurrence relations

$$n_h = n_{h-1} + n_{h-2} + 1, \qquad n_{-1} = 0, \quad n_0 = 1, \tag{6.4}$$

and

$$p_h = p_{h-1} + p_{h-2} + n_h - 1, \quad p_{-1} = p_0 = 0. \tag{6.5}$$

The first is solved by the technique of Section 3.2.2 to yield the general solution

$$n_h = c\left(\frac{1+\sqrt{5}}{2}\right)^h + c'\left(\frac{1-\sqrt{5}}{2}\right)^h - 1.$$

Taking the initial conditions into consideration, we obtain

$$\begin{aligned} n_h &= \left(1 + \frac{2}{\sqrt{5}}\right)\left(\frac{1+\sqrt{5}}{2}\right)^h + \left(1 - \frac{2}{\sqrt{5}}\right)\left(\frac{1-\sqrt{5}}{2}\right)^h - 1 \\ &\approx 1.9\left(\frac{1+\sqrt{5}}{2}\right)^h. \end{aligned} \tag{6.6}$$

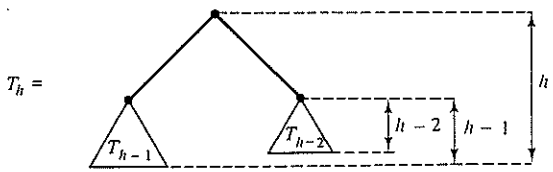


Figure 6.14 The most skewed height-balanced trees are constructed according to a Fibonacci-type rule.

Inverting equation (6.6), we find that

$$h = \frac{1}{\lg[(1+\sqrt{5})/2]} \lg n_h + O(1)$$
$$\approx 1.44 \lg n_h. \tag{6.7}$$

Thus the worst-case search time (i.e., the height of the most-skewed binary tree) in height-balanced binary trees is about 44% longer than the average search time of $\lg n$ in completely balanced binary trees and about the same as the *average* search time in randomly constructed binary trees.

An approximate analysis of the average (rather than the worst-case) successful search time $S_h = p_h/n_h$ in the most-skewed height-balanced binary tree is even more favorable. Dividing recurrence relation (6.5) by n_h gives

$$\frac{p_h}{n_h} = \frac{n_{h-1}}{n_h} \cdot \frac{p_{h-1}}{n_{h-1}} + \frac{n_{h-2}}{n_h} \cdot \frac{p_{h-2}}{n_{h-2}} + 1 - \frac{1}{n_h};$$

that is,

$$S_h = \frac{n_{h-1}}{n_h} S_{h-1} + \frac{n_{h-2}}{n_h} S_{h-2} + 1 - \frac{1}{n_h}.$$

By ignoring the term $-1/n_h$ and using expression (6.6) to approximate n_{h-1}/n_h and n_{h-2}/n_h by $[(1+\sqrt{5})/2]^{-1}$ and $[(1+\sqrt{5})/2]^{-2}$, respectively, we obtain

$$S_h = \left(\frac{1+\sqrt{5}}{2}\right)^{-1} S_{h-1} + \left(\frac{1+\sqrt{5}}{2}\right)^{-2} S_{h-2} + 1. \tag{6.8}$$

This recurrence relation can again be solved by the technique of Section 3.2.2. Using the identity $[(1+\sqrt{5})/2]^{-1} + [(1+\sqrt{5})/2]^{-2} = 1$, we find first that the general solution of the homogeneous recurrence relation corresponding to (6.8) is

$$S_h = c \cdot 1^h + c'\left(\frac{1-\sqrt{5}}{1+\sqrt{5}}\right)^h$$
$$\approx c \quad \text{as } h \to \infty$$

and, secondly, that (6.8) has a particular solution of the form $S_h = \text{constant} \cdot h$, or, more precisely,

$$S_h = \frac{h}{1 + [(1+\sqrt{5})/2]^{-2}}$$
$$= \frac{5+\sqrt{5}}{10} h \approx 0.72h.$$

Hence the solution to (6.8) is

$$S_h = \frac{5+\sqrt{5}}{10} h + c + o(1)$$
$$\approx \frac{5+\sqrt{5}}{10} h \quad \text{as } h \to \infty. \tag{6.9}$$

To compare this average search time for the most-skewed height-balanced binary trees to that of completely balanced binary trees, we substitute (6.7) into (6.9) to obtain

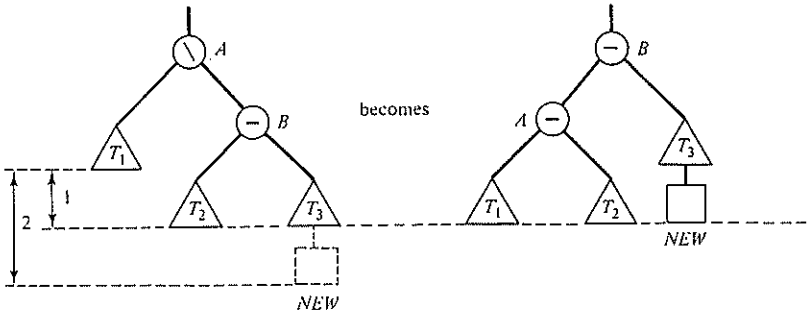
$$S_h \approx \frac{5+\sqrt{5}}{10} \frac{1}{\lg[(1+\sqrt{5})/2]} \lg n_h$$
$$\approx 1.04 \lg n_h. \tag{6.10}$$

Thus the average search time in the most-skewed height-balanced binary trees is only about 4% longer than in completely balanced binary trees. This result, together with the fact that only a small fraction of height-balanced binary trees is badly skewed, supports the previously stated empirical observation that the average search time in height-balanced binary trees, averaged over all such trees, is $\lg n + O(1)$.

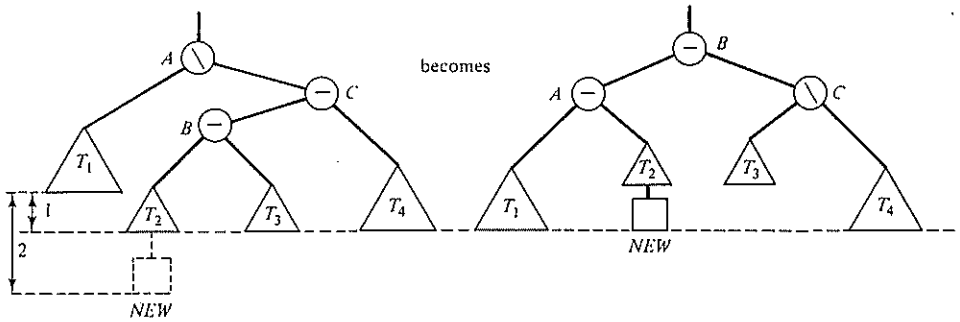
Our analysis establishes that height-balanced binary trees have short search times; now we must show that insertions and deletions can be performed easily while keeping the trees balanced.

Two tree transformations called *rotation* and *double rotation* allow us to restructure height-balanced trees that have been disturbed by an insertion or deletion. Figure 6.15 illustrates these two types of transformation and shows how they restore the balance of a tree disturbed by an insertion. The left parts of Figure 6.15(a) and (b) show a subtree of a height-balanced binary tree in which a new node *NEW* has just been inserted. This insertion may cause the height constraint to be violated at many nodes; among them, consider the lowest node—that is, the root of the subtree shown. (It turns out that height constraints that may have been violated at nodes higher in the tree will be automatically restored.) In the case of the rotation, two nodes with names A and B are shown explicitly in Figure 6.15(a), along with their condition codes \ and −, respectively, which record that before insertion the right subtree of A is of greater height than the left subtree of A, and the heights of the two subtrees of B are equal [the condition code / appears in the mirror image of the rotation shown in Figure 6.15(a)]. Similarly, three nodes labeled A, B, and C are shown explicitly in Figure 6.15(b) for the double rotation.

The reader should verify that the height conditions before and after the transformation hold, as indicated by the dotted lines, and that the *entire* tree is



(a) Rotation.



(b) Double rotation.

Figure 6.15 Examples of a rotation and a double rotation to restore the height balance of a tree disturbed by insertion of a name *NEW*. The double rotation is equivalent to a rotation around node *A*, followed by a rotation around *C*. Both transformations have symmetrical variants.

height balanced after these local transformations. Notice that these transformations preserve the table order of all names in the tree, which is, of course, a prerequisite for performing a search by Algorithm 6.6.

The conditions under which the rotation and double rotation shown in Figure 6.15 (and their symmetrical variants) are applied remain to be specified. This is best done informally.

Insertion Algorithm. After a new name has been inserted by Algorithm 6.8, the path followed from the root to the leaf is traced backward. (Since a binary tree does not ordinarily provide explicit FATHER links, this retracing usually implies that the downward path has been stored on a stack.) The action taken, as each node is traversed along this upward path, depends first on the height condition of

this node (/, –, or \) and, secondly, when this condition is / or \, on the direction of the last step or the last two steps upward along the path. The rules are as follows.

1. If the current node has height condition –, this condition is changed to \ if the last step originated from the right son and to / if it originated from the left son. We continue to follow the path upward unless the current node is the root, in which case the procedure terminates.
2. If the current node has height condition / or \, and the last step originated from the shorter of the two subtrees of the current node, the height condition is changed to –, and the procedure terminates.
3. If the current node has height condition / or \, and the last step originated from the taller of the two subtrees of the current node, then
   (a) if the last two steps were taken in the same direction (both from left sons or both from right sons), an appropriate rotation is performed.
   (b) if the last two steps were taken in opposite directions, an appropriate double rotation is performed.
   In either case, the procedure terminates.

Deletion Algorithm. First a name is deleted according to the algorithm of Figure 6.11. This guarantees that the node to be deleted has at least one empty subtree. If the node to be deleted has exactly one empty subtree, then the height constraint forces the other subtree to consist of a single node, as shown in Figure 6.16. The effect on the height balance of the tree is the same as if a leaf had been deleted; consequently, this is the only case we need to consider. When a leaf is deleted, the subtree consisting solely of this node has lost one unit in height (its height has been changed from 0 to −1). We now trace the path from this deleted leaf up to the root of the tree, checking at each node what effect the shortening of a subtree might have on a larger tree. The action taken at each node along this upward path depends on the height condition of the node before deletion, on the

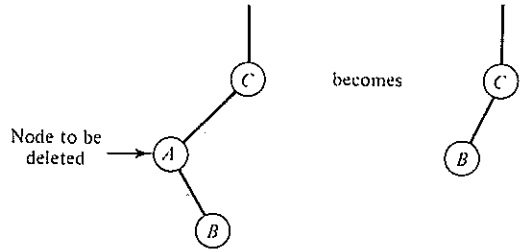
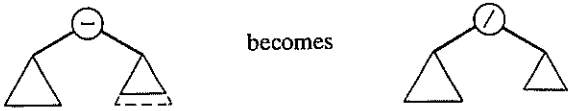


Figure 6.16 Deletion of a node with one empty subtree is reduced to the case of deleting a node with both subtrees empty.

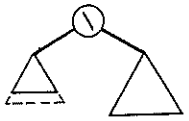
direction of the last step, and, in certain cases, also on the height condition of a son of the node. In the following cases, dotted lines indicate a subtree that has been shortened. In certain cases, a transformation restores the height of a subtree to what it was before deletion; if so, the algorithm terminates, since the height condition of all higher nodes is then unaffected by the deletion. In other cases, the message that a subtree has been shortened must be passed up along the path to the root. In each case, symmetrical variants are *not* shown.

1. If the current node has height condition – then the shortening of the subtree does not affect the height of the tree rooted at the current node. The algorithm terminates.

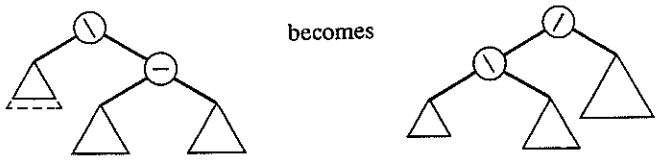


2. If the current node has height condition / or \, and the last step originated from the taller of the two subtrees of the current node, then continue to go upward, passing along the message that the current subtree has been shortened.

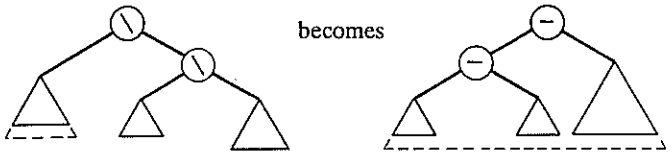
3. If the current node has height condition / or \, and the last step originated from the shorter of the two subtrees of the current node, then the height constraint is violated at the current node. Three subcases and their mirror images are distinguished according to the height condition of the other son of the current node.



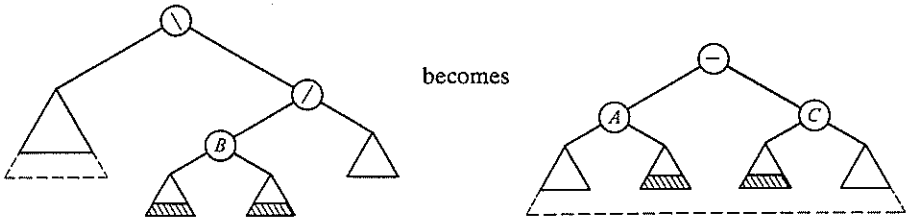
(a) An appropriate rotation restores the height constraint at the current node without changing the height of the subtree rooted at the current node. The algorithm terminates.



(b) An appropriate rotation restores the height constraint at the current node. Continue to go upward, passing along the message that the subtree rooted at the current node has been shortened.



(c) An appropriate double rotation restores the height constraint at the current node. Continue to go upward, passing along the message that the subtree rooted at the current node has been shortened. (The height condition of the node labeled B can be any of /, –, or \ as long as at least one of the two subtrees of B has the height indicated by the shaded area. The height conditions of A and C after the transformation depend on the height of the two shaded subtrees.)



In cases 2, 3(b), and 3(c), the algorithm terminates if the current node is the root.

Notice that this deletion algorithm can require as many as $\lfloor h/2 \rfloor$ transformations, where h is the height of the entire tree (see Exercise 19); this is in contrast to the insertion algorithm, which requires at most one transformation. However, in most cases, the deletion algorithm (and the insertion algorithm as well) will not require retracing the upward path all the way to the root; usually the algorithm terminates after a number of steps that is independent of the height of the tree (see Exercise 20).

6.4.3 *Weight-Balanced Binary Trees*

In the previous section we considered trees that were balanced by constraining the relative heights of the subtrees of each node; here we consider trees that are balanced by constraining the relative number of nodes in the subtrees of each node. The height and the number of nodes are the two most natural measures of the size of a tree; the number of nodes has the advantage that it allows the